Question 1: SIMD

Consider the following code:

```
#define N ... // a large positive value
float A[N];

// initialization of A

for (int i = 1; i < N; i++)
    A[i] += A[i-1];
```

Kfir and Omer want to improve the running time of the loop in this code, but they want to keep the program **single-threaded**. Kfir claims that SIMD instructions cannot be used to improve the performance, while Omer claims that a performance improvement may be possible.

Which of the students is right? Explain your answer.

עומר צודק.

ראשית, למעשה, נשים לב שקוד זה מחשב Prefix sum.

כפי שראינו בתרגול את Prefix sum אפשר לפתור

ב- $O(\log^2(n))$ על ידי שימוש ב- n threads.

כל tread אחראי על בלוק במערך ומוסיף את עצמו

לבלוק מימינו (שנמצא אחריו) בקפיצות של 2.

כלומר תחילה מוסיף לבלוק המרוחק 2^0 ולאחר מכן 2^1

וכן הלאה. סה"כ יש $\log(n)$ איטרציות, ת איטרציה חישובית

מתבצעת ב- $O(1)$.

משום שיש דרישה שלא להשתמש ב-multy threads

נשתמש ב-simd שירוץ כל איטרציה במקביל

ולמעשה יחקה את פעולת ה threds במקביל.

נגדיר: ① וקטור $A = a_0, \ldots, a_{n-1}$

② וקטור $A' = a_0, \ldots, a_{n-1}$

עבור האיטרציה הראשונה נרצה $A' >> 1$, כלומר נקדם
כל איבר ב-2^0 ימינה ואז נרצה $A \leftarrow A + A'$,
כלומר סכמנו לכל איבר את האיבר ה 2^0 מימינו.
כאשר במקביל בזכות יכולות ה-simd ונשיג בכך שבו
למעשה מימשנו בדיוק את האיטרציה הראשונה של prefix sum
באלגוריתם המקורי עם multy threads.

מכאן כל איטרציה נוספת נבצע $\underbrace{A' >> 2^{i-1}}_{*}$
ואחרי $A \leftarrow A + A'$ וכך קיימנו את האיטרציה
ב $O(1)$ כי המימוש הוקטורי שירוץ באופן מקבילי. יש
סה"כ $\log n$ איטרציות. נכונות האלגוריתם נובעת מיידית

מכיוון שהאלגוריתם prefix sum שלמדנו ראינו כי מבצע
את כל האיטרציות ולכן עושה בדיוק, יש אלגוריתם יציב.

יעיל יותר מהאלגורי׳ הסדרתי ב thread יחיד
בעזרת שימוש ב simd.

* כל איטרציה מבצעת $A' >> 2^{i-1}$ ולא 2^{i} מאחר
שב roll מתווספות לקפיצות הקודמות שבוצעו וזו 2^{i-1}

ולכן סוף הקפיצה באיטרציה היא סכימה $2^{i-1}+2^{i-1}=2^{i}$.

Question 2: Binary Search with SIMD

Below is an algorithm for binary search in a sorted array.

```
Sequential Binary Search(A, x, n):
    left ← 0
    right ← n - 1
    while (true)
        if left > right
            return false
        mid ← $\lfloor (left + right)/2 \rfloor$
        if A[mid] == x
            return true
        if A[mid] > x
            right ← mid - 1
        else
            left ← mid + 1
```

Can this algorithm be made more efficient using SIMD operations? If yes, suggest a way. If not, explain why.

כן.

נגדיר B וקטור מאורך רוחב $|A| = n$ שכל איבריו

הם x.

<u>אלגוריתם,</u>

① אתחל B

② $C = (A == B)$ *

③ $C \leftarrow prefix\_sum(C)$

④ אם $1 \le C_{n-1}$ אחזיר true, אחרת → false.

* הכוונה בפעולה ② הינה ביצוע mask
ומחזיר 0 או 1 עבור פעולת השוואה בין

A_i ל B_i. C_i למעשה 0 אם אין שוויון, כלומר
$X \neq A_i$ ואילו C_i שווה 1 אם $X = A_i$.

נכונות: אם $x \in A$ אז יש $X_i = A_i$ ולפי הגדרה

C נקבל $C_i = 1$ מכאן שעבור את ③ נקבל

מערך prefix sum ואילו C_{n-1} מכיל את
C_i לכן ודאי $1 \leq C_{n-1}$.

אילו $x \notin A$ לכל i נקבל $C_i = 0$ ולשעבור prefix sum

$C_{n-1} = \sum_{i \in (n-2)} C_i = \sum_{i \in (n-2)} 0 = 0$, כלומר $1 > C_{n-1}$

מבחינת זמן הריצה נשים לב שפעולה 1 ו-2 מיידית,
כלומר בזמן הקבוע. פעולה 3 היא ביצוע prefix sum

אשר ראינו במימוש מסויים k על ידי simd
ופעולה 4 [illegible] מבוצעת בזמן הקבוע.

סה"כ מקבלים מימוש ב-simd שמאפשר את זמן הסיבוכיות
כנדרש.